

Multi-Store Database Review & Improvement PRD

A beginner-friendly explanation of the current database, identified risks, long-term consequences, recommended solutions, and implementation plan.

Scope

Shopify-to-Supabase monthly analytics sync for the latest 18 months. This document reviews storage and multi-store safety only. Existing live reports continue to read ShopifyQL until reconciliation and approval are complete.

Document	Value
Purpose	Approve a safe multi-store database design before more stores are onboarded
Current status	One store synced; database is not yet the source used by live reports
Audience	Business owner, product owner, developer, and implementation reviewer
Decision needed	Approve the priority fixes and rollout sequence in this PRD

OUTCOME FIRST

1. Executive Summary

Overall assessment

The current design is a good monthly analytics foundation and the first store reconciles correctly with ShopifyQL. It is suitable for controlled testing. Before adding many stores or making Supabase the live reporting source, four safeguards should be implemented: enforce store-product ownership, make retention store-specific, replace a month atomically, and centralize all reads behind authenticated store scope.

What is already working well

1. Store identity is separated from product identity. The same Shopify product number can exist in two stores without colliding because the product is uniquely identified by **store + Shopify product ID**.
2. Monthly facts are compact. Product text is stored once, while monthly rows mainly store numbers. This is appropriate for an 18-month retention model.
3. The sync uses batching, limited concurrency, retries, progress tracking, and a targeted retry for failed months.
4. The first store completed reconciliation for selected months with zero identifiable product mismatches and zero store-total mismatches.

Main decision

Proceed with Supabase, but complete the critical multi-store integrity work before onboarding Store 2. After that, improve metric naming and historical accuracy in phases. This keeps the project moving without requiring a redesign.

Important clarification about product tags

The absence of `store_id` in `audit_product_tags` is intentional and safe. The table stores the database's internal `product_id`, not the Shopify product number. That internal product belongs to exactly one `audit_products` row, and that row contains `store_id`. Store ownership is therefore obtained through the product relationship.

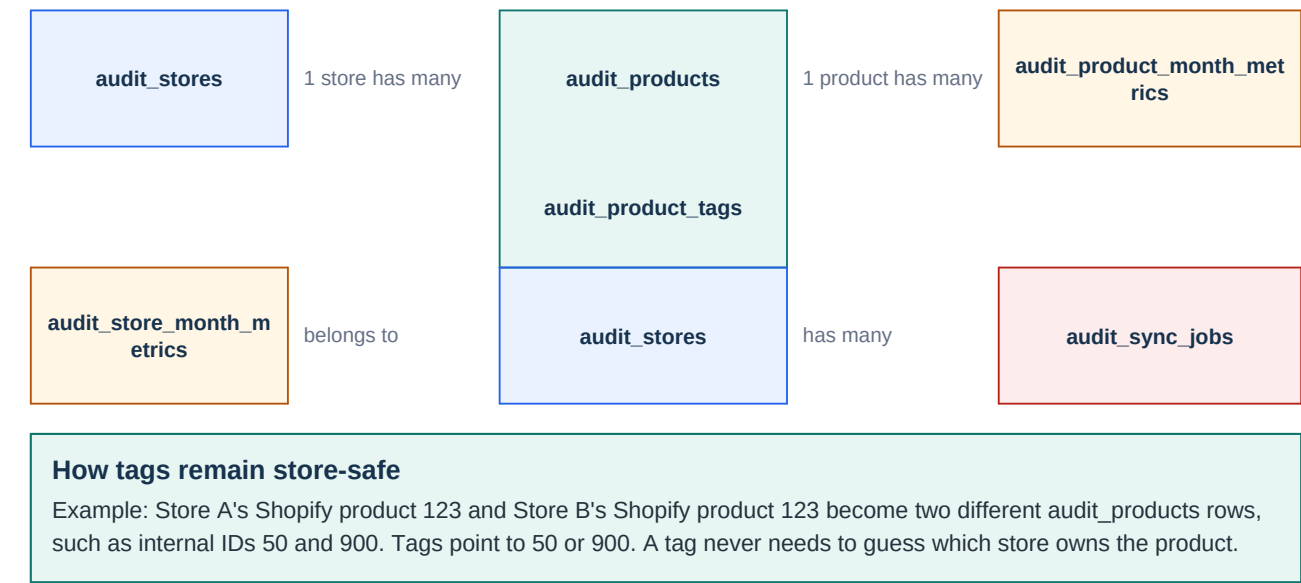
HOW IT WORKS TODAY

2. Current Database Working and Structure

2.1 Simple mental model

Step	What happens	Why it exists
1. Store	One audit_stores row identifies each Shopify shop.	Separates one brand/store from another.
2. Product catalog	audit_products stores current title, handle, type, status, image, and Shopify product ID.	Avoids repeating product text every month.
3. Tags	audit_product_tags stores many tags against the internal product row.	Allows filtering without repeating tags in monthly facts.
4. Product -month facts	audit_product_month_metrics stores one row per store, product, and month.	Powers product audit and cohort calculations.
5. Store-month summary	audit_store_month_metrics stores one overall row per store and month.	Provides validation and fast high-level totals.
6. Sync progress	audit_sync_jobs records status, date range, counts, and errors.	Makes long imports observable and retryable.

2.2 Relationship map



NEW SECTION: FROM SHOPIFY SOURCE TO REPORT SCREEN

2.3 The Complete Data Journey - Beginner Version

The most important idea

There are currently **two separate data journeys**. The live reports still ask Shopify for data when a report is opened. Separately, the new sync copies 18 months of monthly data into Supabase so it can be checked and prepared for future use. Supabase is not yet feeding the live Product Audit or New Arrival reports.

Simple meanings of the main words

Word	Meaning in very simple language	Example in this app
Source	The original place where the information comes from.	Shopify is the source of product, inventory, sales and session information.
ShopifyQL	A question sent to Shopify asking it to calculate and return analytics rows.	Give inventory and sales for each product for August.
Server	The private part of the app that talks to Shopify and prepares the report.	It checks which store is signed in and joins the returned rows.
Database	Organized tables where information can be saved for later use.	Supabase stores one product-month row for each product and month.
Cache	A temporary saved answer used to avoid asking Shopify the same question again immediately.	A finished historical month can be reused when the same report is requested again.
Batch / chunk	A large job divided into smaller pieces so it is less likely to fail.	Inventory is split into smaller date ranges and the results are combined.
Retry	If Shopify is temporarily busy, the app waits and tries again.	A rate-limited monthly query is retried instead of failing immediately.
Transform	Cleaning, matching or calculating data before showing or saving it.	Match a Shopify product ID to its title, then calculate CR %.
Embedded app	The report page shown inside the Shopify Admin window.	Product Audit and New Arrival Analysis.

Think of it like a restaurant

Shopify is the storeroom holding the ingredients. ShopifyQL is the order slip asking for specific ingredients. The app server is the kitchen that combines them. The browser is the waiter, and the report is the prepared dish shown to the user. Supabase is a labelled pantry being prepared so future dishes can be served faster and more consistently.

WHAT HAPPENS WHEN THE USER OPENS OR CHANGES A REPORT

2.4 Current Live Report Pipeline

1. The user opens Product Audit or New Arrival Analysis inside Shopify Admin.
2. Shopify confirms which store is signed in. The app does not rely on a store name typed by the user.
3. The page URL tells the server the selected start date, end date, grouping and filters.
4. The server loads the current Shopify product catalog through Shopify Admin GraphQL. This provides names, handles, types, statuses, images and tags.
5. The server sends analytics questions to ShopifyQL. Product Audit asks for product landing sessions, product sales, store unique orders and inventory. New Arrival also asks for store sales and the special first-cohort lookback data.
6. Large inventory requests are divided into smaller date chunks. If a chunk reaches Shopify's row limit, it is divided again. Temporary rate-limit errors are retried with waiting time.
7. The server converts Shopify's returned rows into one consistent format, because Shopify may return different row shapes.
8. Rows are matched using Product ID. Landing sessions are matched using the product URL handle. The app combines catalog details, inventory, sessions and sales.
9. Report rules calculate totals, New Arrival cohorts, percentages, CR %, filters, sorting and pagination.
10. React Router sends the prepared result to the browser. The old report body is replaced by a loader while the new result is being prepared.
11. The browser draws the tables and cards inside the Shopify embedded app. Export uses the same prepared report data.

Where the live report data comes from today

Shopify is still the source of truth. The app may reuse a safe cached result for a completed historical period, but if the required answer is not cached, it asks ShopifyQL. The new Supabase monthly tables are not yet read by these report pages.

Data needed	Original Shopify source	How it joins the report
Product details	Shopify Admin GraphQL product catalog	Shopify Product ID
Starting/ending inventory	ShopifyQL inventory analytics	Product ID + date
Sales, units and product orders	ShopifyQL sales analytics	Product ID + date
Store unique orders	ShopifyQL store-level sales analytics	One total for the selected range
Landing sessions	ShopifyQL sessions analytics	Landing page path / current product handle
Product image and website URL	Current product catalog and product handle	Shown with the product row

HOW THE 18-MONTH DATABASE COPY IS CREATED

2.5 Supabase Monthly Sync Pipeline

1. The sync starts only for an authenticated Shopify store.
2. The app creates or updates that store in `audit_stores`.
3. The current product catalog is fetched once and stored in `audit_products`. Tags are stored in `audit_product_tags`.
4. The latest 18 months are listed from newest to oldest.
5. For each month, the app asks ShopifyQL for product inventory, product sales, product landing sessions and store totals.
6. Only a small number of Shopify questions run together. This reduces rate-limit pressure.
7. Temporary failures are retried. If a month remains incomplete, that month is not written as if it were complete.
8. Shopify Product IDs are matched to the internal `audit_products` IDs. Session paths are matched to product handles.
9. The app prepares compact monthly rows. Text such as title is stored once; changing numbers are stored by month.
10. Rows are written to `audit_product_month_metrics` in batches of 500. One overall monthly summary is stored in `audit_store_month_metrics`.
11. `audit_sync_jobs` records progress, successful months, failures and row counts. A failed month can be retried separately.
12. After approval in a future phase, reports can read these prepared rows instead of repeating every historical ShopifyQL query.

Saved table	What is saved	Why
<code>audit_stores</code>	Store identity, currency and status	Keeps every store separate
<code>audit_products</code>	Current product details	Avoids repeating text for every month
<code>audit_product_tags</code>	Tags linked to the internal product	Supports product-tag filters
<code>audit_product_month_metrics</code>	Monthly inventory, sessions, orders, units and sales	Main monthly facts for reporting
<code>audit_store_month_metrics</code>	One monthly store summary	Fast checks against Shopify totals
<code>audit_sync_jobs</code>	Progress, dates and errors	Shows whether the import completed safely

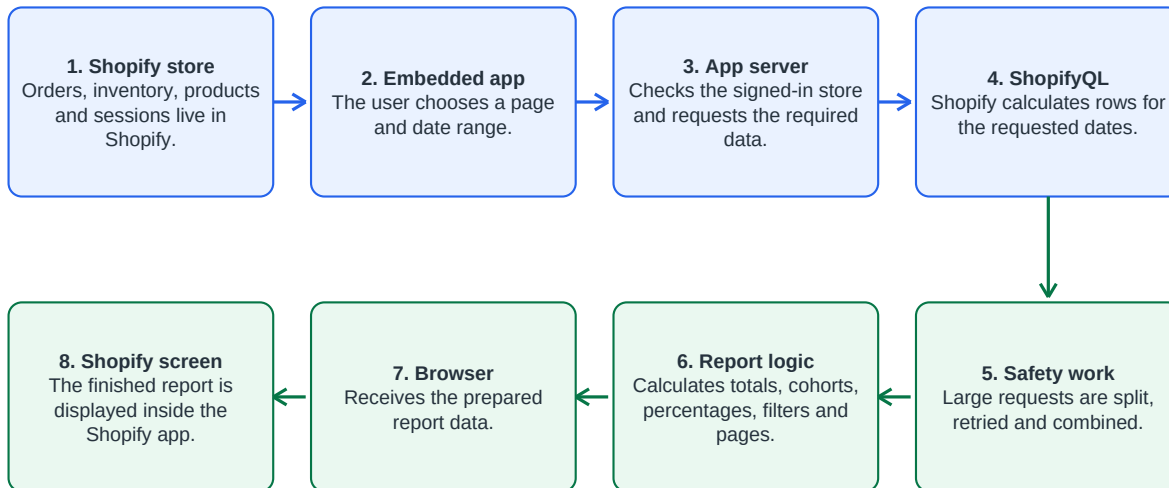
Why monthly storage is smaller

Product names and tags are not copied 18 times. The catalog text is stored once. Each monthly row mainly stores IDs, a month and numbers. This is why the database can hold more stores than a daily full-product copy.

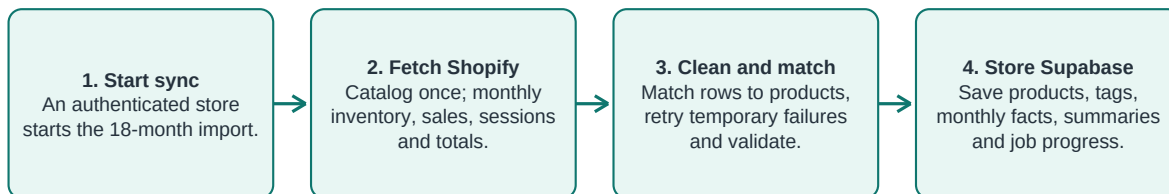
THE TWO DATA JOURNEYS SHOWN TOGETHER

2.6 End-to-End Flowchart

A. What happens when a person opens a live report today



B. Separate background preparation flow for Supabase



Important: these two lanes are not connected for live reporting yet
Supabase is currently a checked copy used for comparison. Product Audit and New Arrival Analysis still read live ShopifyQL. A later approved change will connect reports to Supabase.

FROM PREPARED ROWS TO THE USER'S SCREEN

2.7 How the Data Finally Appears in Shopify

What the browser receives

1. The server sends prepared report data, not raw Shopify passwords or Supabase service keys.
2. The response contains rows, totals, date information, errors/warnings and freshness details needed by that screen.
3. The browser uses React components to draw the date selector, metric cards, tables, filters, pagination and exports.
4. When the date changes, the URL changes and the route loader runs again. A full loader replaces the previous report body so old numbers are not mistaken for new numbers.
5. Client-only actions, such as switching some already-loaded views or paging through prepared results, can be faster because they do not always require another Shopify request.

What will change after a future database-read rollout

Step	Today	Future after approval
Historical data source	ShopifyQL or existing analytics cache	Supabase monthly rows for stored periods
Fresh/current period	ShopifyQL	Likely ShopifyQL for the latest incomplete period, then save/refresh
Report calculations	App server and New Arrival engine	Same approved calculation rules, using database rows
Shopify screen	React tables inside embedded app	Same user experience; only the source behind it changes

Safe rollout principle

The user should not see a different answer just because the source changes from live ShopifyQL to Supabase. Before switching, the same months and products must be compared. Any difference must be explained and approved.

WHAT EACH STORED FIELD MEANS

3. Table and Metric Dictionary

Table	Row meaning	Main content	Purpose
audit_stores	One row per Shopify store	shop_domain, currency, status, lifecycle dates	Tenant identity
audit_products	One row per product per store	current title, handle, type, status, image, created date	Current catalog snapshot
audit_product_tags	One row per internal product and tag	product_id + tag	Current product tags
audit_product_month_metrics	One row per store + product + month	inventory, sessions, orders, units, sales	Monthly product facts
audit_store_month_metrics	One row per store + month	active products, inventory, unique orders, sessions, sales	Store-level monthly check
audit_sync_jobs	One row per sync attempt	status, range, counts, details, errors	Progress and operations

Important metric distinctions

Metric	Plain-language meaning
starting_inventory	Inventory at the start boundary for the product/month. It can be negative in raw ShopifyQL data.
ending_inventory	Inventory at the end boundary for the product/month. It can be negative in raw ShopifyQL data.
first_day_in_inventory	A diagnostic/product-audit field. The New Arrival cohort rule intentionally does not use the first inventory date.
orders (product)	Orders containing that product. Adding this across products can count the same customer order more than once.
unique_orders (store)	Distinct orders for the whole store. This is not the sum of product orders.
landing_sessions	Sessions attributed to a product landing page and matched through a product handle.
sales * _minor	Money stored in the smallest currency unit, such as paise or cents, to avoid decimal rounding errors.
active_products	Products meeting the report activity rule, not variants or SKUs.

NA Inventory % denominator

For a displayed period, NA Inventory is the positive inventory contributed by products in the selected new-arrival cohort. NA Inventory % divides this by the positive inventory of all active products in the same displayed group and period. Negative inventory is treated as zero for this percentage. The store-month starting_inventory and ending_inventory fields are raw validation totals and are not automatically the denominator.

END-TO-END PROCESS

4. Current Shopify-to-Supabase Sync Workflow

1. A user opens the authenticated Shopify development app and starts the analytics sync.
2. The server identifies the shop and currency, then creates or updates its `audit_stores` row.
3. The app blocks another overlapping sync for that same store.
4. The full current product catalog is fetched once. Products and tags are written to Supabase.
5. The latest 18 months are processed from newest to oldest.
6. For each month, ShopifyQL queries retrieve inventory, product sales, product landing sessions, and store-level totals.
7. At most two ShopifyQL queries run concurrently. Rate-limited or temporary failures are retried with delay.
8. Rows are matched to the internal product record. Product-month rows are upserted in batches of 500.
9. The store-month summary is upserted, and `audit_sync_jobs` records progress, row counts, and errors.
10. A failed month can be retried separately. Old months are removed according to the 18-month retention rule.

Current first-store result

Measure	Result
Store	d2c-apps.myshopify.com (INR)
Products	4,003
Product-tag rows	10,983
Product-month facts	20,268
Store-month summaries	18 months: Apr 2025 to Sep 2026
Reconciliation	Selected May, Jul, and Aug 2026 checks: zero mismatches for identifiable products and store totals
Current source for live reports	ShopifyQL. Supabase has not replaced live reads.

What reconciliation proves

It shows that the current extraction and mapping can reproduce the checked ShopifyQL results for the first store. It does not by itself prove multi-store isolation, atomic refresh behavior, historical classification accuracy, or future timezone/currency safety. Those are addressed below.

FIX BEFORE STORE 2

5. Critical Multi-Store Issues and Recommendations

1. Store-product ownership is not enforced as one pair

CRITICAL

Issue in simple language: A monthly fact stores both store_id and product_id. Each value is valid separately, but the database does not currently prove that the product belongs to that same store.

Example: A bug could write Store A's store_id with Store B's product_id. Both foreign keys pass, even though the pair is wrong.

Long-term / multi-store consequence: Data can leak into the wrong store's totals, cohorts, or exports. The error may remain hidden because every referenced row exists.

Recommended solution: Add a database-level composite relationship between product_id and store_id. Keep store_id on the fact table for fast filtering, but make an incorrect pair impossible to save.

2. Retention cleanup is not store-scoped

CRITICAL

Issue in simple language: The cleanup function currently removes old monthly rows based only on month, not on the store being synchronized.

Example: If Store A completes a sync with a later retention boundary, its cleanup can delete older rows that still belong to Store B.

Long-term / multi-store consequence: One customer's routine sync can silently remove another customer's historical data. The risk grows immediately when a second store is added.

Recommended solution: Require store_id in the cleanup function and include it in every delete condition. Add a test proving a Store A cleanup leaves Store B unchanged.

5. Critical Multi-Store Issues and Recommendations (continued)

3. A month is upserted, not fully replaced

CRITICAL

Issue in simple language: Upsert updates returned rows and adds new ones, but it does not remove an old row that Shopify no longer returns.

Example: A product appeared in June during the first sync. A corrected ShopifyQL result later excludes it. Re-running June updates other rows, but the old product row can remain.

Long-term / multi-store consequence: Stale facts inflate counts, cohorts, inventory, or sales after corrections. Re-running a sync does not guarantee the database exactly matches Shopify.

Recommended solution: Use a transaction or Supabase database function: load and validate the complete store-month result, then replace that store-month atomically. If the new load fails, keep the previous good month.

4. Service-role access depends on perfect application filtering

CRITICAL

Issue in simple language: Browser roles are blocked, which is good. The server uses the service-role key, which can read every store and bypasses row-level security.

Example: If a report endpoint accepts `store_id` from the browser and forgets one filter, it could return another store's rows.

Long-term / multi-store consequence: As more pages and developers are added, one missed filter can become a serious tenant-isolation and privacy incident.

Recommended solution: Create one server-side data-access layer. It must derive store identity from the authenticated Shopify session, apply `store_id` automatically, and never trust a browser-provided `store_id`. Add cross-store isolation tests.

PREVENT MISLEADING REPORTS

6. Metric and Data Meaning Issues

5. The data is product-level, although some labels say SKU

HIGH

Issue in simple language: The stored key is Shopify Product ID. Inventory is aggregated across variants and locations, so one row represents a product, not one SKU/variant.

Example: A shirt with 5 sizes is one product row even though Shopify may have 5 SKUs.

Long-term / multi-store consequence: Users can misread NA SKU %, SKU count, or active SKU metrics as variant-level measures and make incorrect merchandising decisions.

Recommended solution: Choose the intended grain. If product-level is correct, rename definitions and labels to Product. If true SKU analysis is required later, add a variant-level model rather than mixing grains.

6. Raw negative inventory and clamped NA inventory use different rules

HIGH

Issue in simple language: Store summaries retain raw inventory, including negative values. NA Inventory % treats negative inventory as zero.

Example: For Aug 2026, raw starting inventory was 912,771; after treating negatives as zero it was 914,020, a difference of 1,249. The audit found 163 product-month rows with negative inventory.

Long-term / multi-store consequence: Two screens can show different inventory totals while both are technically following their own rules. Users may think the database is wrong.

Recommended solution: Name raw values clearly (for example, raw_starting_inventory) and either add explicit non-negative fields or calculate them consistently in a documented reporting layer.

6. Metric and Data Meaning Issues (continued)

7. Product orders and store unique orders are not additive

HIGH

Issue in simple language: Product orders count an order for each product it contains. Store unique orders count the customer order once.

Example: One order containing Product A and Product B contributes 1 to each product, but only 1 to store unique orders. Summing product orders gives 2.

Long-term / multi-store consequence: A report that sums product orders can overstate order volume and distort conversion metrics.

Recommended solution: Keep both metrics, but use exact names such as Product Orders and Store Unique Orders. Block or warn against summing product orders as a store total.

8. Landing sessions depend on the current product handle

HIGH

Issue in simple language: Product landing-page sessions are matched using the product handle. The catalog stores only the current handle, while historical URLs may use an older handle.

Example: A product was /products/pink-dress in May and renamed to /products/rose-dress in August. May sessions may no longer match the current product record.

Long-term / multi-store consequence: Historical landing sessions and CR % can be understated, especially for renamed products. Store-level sessions built from matched products can also be low.

Recommended solution: Store handle history with valid-from and valid-to dates, and keep an unmatched-session bucket. Where Shopify provides a direct store-level session total, store it separately for reconciliation.

6. Metric and Data Meaning Issues (continued)

9. Product type, tags, title, and handle are current snapshots

HIGH

Issue in simple language: The product table stores today's catalog values. Historical monthly facts point to that current row.

Example: A product classified as Dresses in January changes to Occasion Wear in September. January may now appear under Occasion Wear if a report joins to the current product table.

Long-term / multi-store consequence: Historical cohort composition can move when catalog metadata changes, making old exported results hard to reproduce.

Recommended solution: Decide explicitly between current classification and historical classification. For reproducible history, store monthly type/handle snapshots or effective-dated history. For current classification, label reports clearly.

10. Removed products do not have a clear lifecycle state

HIGH

Issue in simple language: Historical product rows remain, which is useful, but there is no catalog_present, last_seen_at, or deleted_at field to explain whether a product still exists in Shopify.

Example: A deleted product may retain an old ACTIVE status in the database because it is no longer returned to be updated.

Long-term / multi-store consequence: Users can mistake stale metadata for a live product. Tags may also disappear during a refresh, reducing historical context.

Recommended solution: Add lifecycle fields and mark products missing from a successful full catalog sync. Preserve historical facts while clearly distinguishing current catalog presence.

7. Operational and Future-Proofing Issues

11. Tag refresh is not atomic

HIGH

Issue in simple language: The current pattern deletes a store's tags and then inserts the refreshed set. If insertion fails after deletion, the old correct tags are already gone.

Example: A temporary network error occurs between delete and insert. Product facts survive, but tag filters become empty or incomplete.

Long-term / multi-store consequence: Reports can suddenly lose product-tag classifications until the next successful full sync.

Recommended solution: Refresh tags inside one database transaction, or load into staging first and swap only after validation.

12. Currency is safe per store but unsafe across stores

HIGH

Issue in simple language: Money is stored in minor units and each store has a currency code. That works for one store. It does not make INR, USD, and GBP directly comparable.

Example: Adding INR 10,000 and USD 500 produces a number with no valid currency meaning.

Long-term / multi-store consequence: A future agency-level dashboard could show incorrect revenue if it sums stores without conversion. A later currency change inside one store also needs history.

Recommended solution: Never sum currencies without conversion. Store currency with each monthly summary (or maintain currency history), and require an explicit reporting currency plus exchange-rate policy for cross-store totals.

7. Operational and Future-Proofing Issues (continued)

13. Month boundaries use the app server timezone

HIGH

Issue in simple language: The latest completed day/month is currently calculated from the application server's clock rather than each Shopify store's timezone.

Example: At midnight in India, a US store may still be on the previous calendar date.

Long-term / multi-store consequence: First/last-day inventory and current-month ranges can shift by one day for stores in other timezones.

Recommended solution: Store each shop's IANA timezone and calculate date boundaries in that timezone. Add boundary tests around midnight and daylight-saving changes.

14. Sync progress fields can be misread

MEDIUM

Issue in simple language: attempts currently describes job-level attempts but can remain 1 even when individual ShopifyQL queries retry. rows_written mixes processed/upserted operations and is not the same as newly inserted database rows.

Example: A job can show attempts = 1 even though one monthly query retried several times.

Long-term / multi-store consequence: Operations cannot accurately identify rate-limit pressure, write amplification, or how much data actually changed.

Recommended solution: Track job_attempt, query_retry_count, rows_processed, rows_inserted, rows_updated, and rows_deleted separately. Keep a short human-readable error summary.

LOWER-RISK BUT WORTH DECIDING

8. Additional Design Findings

#	Topic	Finding	Recommended decision
15	Shopify IDs in JavaScript	Current IDs are below JavaScript's safe integer limit, but a 64-bit ID could eventually exceed it.	Treat Shopify IDs as text end-to-end. Keep internal database IDs numeric.
16	Unattributed source rows	ShopifyQL can return a row without a usable Product ID; one such May row was intentionally excluded.	Store an explicit unattributed bucket for reconciliation instead of silently mixing it with products.
17	Store inventory summary purpose	Store-month starting/ending inventory is currently a validation field and is not required for NA Inventory %.	Rename/document it as raw validation inventory, or remove it if it has no approved consumer.
18	first_day_in_inventory purpose	The field is useful for audit diagnostics, but the NA cohort rule does not use the first historical inventory date.	Document this separation and prevent accidental reuse in cohort assignment.
19	active_products naming	The count represents active products based on inventory/sales, not variants or SKUs.	Use Active Products consistently unless a variant table is introduced.

Current data-quality snapshot

At the time of review: 1 store, 4,003 products, 10,983 tags, 20,268 product-month facts, 18 store-month summaries, and 3 sync jobs. There were zero orphan tags, zero store/product mismatches, and zero case-variant duplicate tags. The absence of current mismatches is positive, but constraints are still needed to prevent future bad writes.

PRIORITIZED IMPLEMENTATION

9. Recommended Plan of Action

Phase	Goal	When	Main work
Phase 0	Decision lock	1-2 working sessions	Confirm product vs SKU grain; current vs historical classifications; raw vs clamped inventory; timezone and currency rules.
Phase 1	Critical multi-store safety	Before Store 2	Composite store-product constraint; store-scoped retention; authenticated repository; atomic month replacement; isolation tests.
Phase 2	Metric clarity and observability	Next	Rename product/store order metrics; clarify Product labels; explicit inventory semantics; improve sync counters and unmatched sessions.
Phase 3	Historical reliability	After stable sync	Product lifecycle fields; handle history; historical type/tag strategy; store timezone; currency history.
Phase 4	Controlled rollout	Progressive	Re-sync Store 1; reconcile; add Store 2; verify isolation; then expand 1 -> 5 -> 20 -> 50 stores.

Detailed execution checklist

1. Create a migration backup and rollback plan before changing constraints.
2. Add the composite ownership constraint and run a query proving zero existing violations.
3. Make every retention/delete function require store_id and add a two-store automated test.
4. Create a transactional replace_store_month function or staging workflow.
5. Centralize Supabase reads/writes in server-only modules that derive store from Shopify authentication.
6. Add contract tests for product/store ownership, stale-row removal, retry safety, and failed tag refresh.
7. Add approved metric names and definitions to the calculation logic documentation.
8. Re-run the 18-month sync for Store 1 and compare selected products and months with ShopifyQL.
9. Onboard a controlled Store 2 whose Shopify product numbers may overlap; prove no collision or cross-read.
10. Monitor database size, egress, ShopifyQL rate limits, failed months, retry counts, and end-to-end sync duration.

HOW WE KNOW THE DESIGN IS READY

10. Acceptance Criteria

ID	Acceptance requirement
AC-01	Database rejects a monthly fact whose product belongs to another store.
AC-02	Syncing or cleaning Store A cannot insert, update, or delete Store B data.
AC-03	Re-syncing a corrected month removes stale rows and either completes fully or preserves the previous good month.
AC-04	Every application read derives store identity from the authenticated Shopify session.
AC-05	Product vs SKU, raw vs clamped inventory, product orders vs unique orders, and NA Inventory % are documented and use final names.
AC-06	Selected-month reconciliation against live ShopifyQL produces zero unexplained mismatches.
AC-07	Two stores can contain the same Shopify product ID without collision.
AC-08	A failed tag refresh leaves the last successful tag set intact.
AC-09	Different currencies are never summed without an explicit conversion method.
AC-10	Month/date boundaries pass tests in store timezones, including midnight and daylight-saving cases.
AC-11	Sync monitoring distinguishes job attempts, query retries, processed rows, and actual database changes.
AC-12	Supabase remains in comparison mode until the business owner approves switching a live report.

Go-live rule

Do not switch Product Audit or New Arrival Analysis to database reads simply because the sync completes. Switch only after Phase 1 is complete, the relevant metric decisions are approved, and reconciliation passes for more than one store.

SHORT APPROVAL LIST

11. Decisions Required from the Product Owner

ID	Decision	Question	Recommendation
D1	Reporting grain	Keep product-level reporting and rename SKU labels, or invest in variant/SKU-level storage?	Recommend product-level for current scope.
D2	Historical classification	Should old months show today's type/tags or the type/tags that existed then?	Recommend historical snapshots for reproducible cohorts.
D3	Inventory meaning	Should general inventory totals remain raw, or should user-facing totals clamp negatives to zero?	Recommend store both meanings with explicit names.
D4	Cross-store revenue	Will there be agency totals across currencies?	If yes, approve reporting currency and FX source.
D5	Deleted products	Should deleted products remain searchable in historical reports?	Recommend yes, with clear lifecycle labels.
D6	Unmatched sessions	Show an Unattributed bucket or only matched products?	Recommend retain and report unmatched totals.

Recommended approval

Approve Phase 1 immediately, keep Supabase in shadow/comparison mode, and resolve D1-D3 before report migration. D4-D6 can be finalized during Phase 2 without blocking the critical safety work.

Final recommendation

The architecture can support the planned multi-store model without an external redesign. The safest and most efficient path is to strengthen tenant integrity first, make refreshes atomic, then improve metric clarity and historical fidelity. This preserves the work already completed and creates a reliable path toward 50 stores.